Knapsack problems

Outline

Overview of the family

The 0/1 knapsack problem

The subset sum problem

Knapsack problems with several copies

Knapsack problems with several resources

Knapsack problems with several knapsacks

Knapsack problems with classes

Typical mistakes

Overview of the family

Introduction: knapsack problems

- A knapsack problem selects items of maximum total profit while respecting capacities.
- The variants differ in the copies of an item, the capacities and knapsacks, and the classes of items.
- This chapter introduces the family; later chapters model and solve its members.

The common data

- Items $\mathscr{I} = \{1, \ldots, n\}$ have weight $w_j \in \mathbb{Z}_{\geq 1}$ and profit $p_j \in \mathbb{Z}_{\geq 1}$.
- A solution selects items, or a number of copies, subject to capacity constraints.
- The objective is to maximize total profit.

variant	difference	solution
KP	one copy, one knapsack	subset
SSP	$p_j = w_j$, exact target	subset
BKP / UKP	bounded / unlimited copies	integer vector
MDKP	several capacities	subset
MKP	several knapsacks	assignment
MCKP / KPS	classes of items	subset

Summary

- The same selection logic appears in budgeting, loading, advertising and investment.
- The form of the decision determines the candidate solutions and the algorithms that apply.

The 0/1 knapsack problem

Introduction: the 0/1 knapsack

- Each item is selected once or not selected.
- This is the reference problem for the family and the running instance of the chapter.
- We identify feasible subsets, a primal bound and a first dual bound.

Definition 1: 0/1 knapsack problem

Given capacity c, items $\mathscr{I} = \{1, \ldots, n\}$ with profits p_j and weights w_j, select a subset $S \subseteq \mathscr{I}$ such that $\sum_{j \in S} w_j \leq c$ and with maximum total profit:

$$\widetilde{S} \in \arg\max_{S \subseteq \mathscr{I}} \left\{ \sum_{j \in S} p_j : \sum_{j \in S} w_j \leq c \right\}.$$

Example

For $c = 9$, let

j	1	2	3	4	5	6
p_j	6	5	8	9	6	7
w_j	2	3	6	7	5	9

.

The subset $\{1, 4\}$ has weight $2 + 7 = 9$ and profit $6 + 9 = 15$.

Candidates, bounds and optimality

- There are $2^6 = 64$ candidate subsets; only 13 are feasible.
- The solution $\{1, 4\}$ gives the primal bound $\mathrm{LB} = 15$.
- No feasible subset has three items: the three lightest weigh $2 + 3 + 5 > 9$.

Observation 1: cardinality bound

If every feasible subset has at most $\bar{k}$ items, the sum of the $\bar{k}$ largest profits is a dual bound.

Here $\bar{k} = 2$, hence $\mathrm{UB} = 9 + 8 = 17$ and $15 \leq z(I) \leq 17$.

Summary

- In a 0/1 knapsack each item is an IN/OUT decision.
- A feasible solution gives a primal bound; a cardinality restriction gives a simple dual bound.
- Stronger bounds will come from relaxations and specialized algorithms.

The subset sum problem

Introduction: subset sum

Subset sum asks whether a target can be reached exactly. It is a special knapsack problem and returns in dynamic programming and complexity.

Definition 2: subset sum problem

Given a target c and weights w_j, find a subset $S \subseteq \mathscr{I}$ such that

$$\sum_{j \in S} w_j = c.$$

Equivalently, maximize $\sum_{j \in S} w_j$ subject to $\sum_{j \in S} w_j \leq c$.

Summary

- Subset sum is the KP with $p_j = w_j$.
- A feasible subset with total weight c is a certificate of optimality.

Knapsack problems with several copies

Introduction: copies

- Some decisions specify how many copies of each item to take.
- Candidate solutions are integer vectors, so identical copies are not counted repeatedly.

Definition 3: bounded and unbounded knapsack

For BKP, item j has at most b_j copies:

$$\max \left\{ \sum_j p_j x_j : \sum_j w_j x_j \leq c,\ 0 \leq x_j \leq b_j,\ x_j \in \mathbb{Z} \right\}.$$

For UKP, the upper bounds b_j are removed.

Summary

- Bounded and unbounded knapsack replace binary choices by non-negative integer multiplicities.
- Capacity makes every instance finite; in UKP, $x_j \leq \lfloor c/w_j \rfloor$.

Knapsack problems with several resources

Introduction: several capacities

An item can consume several scarce resources, such as weight, volume and energy. A feasible subset must respect every capacity.

Definition 4: multidimensional knapsack problem

Given k capacities c_d and consumptions w_{dj}, choose $S \subseteq \mathscr{I}$ maximizing $\sum_{j \in S} p_j$ such that

$$\sum_{j \in S} w_{dj} \leq c_d, \qquad d = 1, \ldots, k.$$

Summary

MDKP is still an IN/OUT problem, but every chosen item consumes several resources and every capacity must be respected.

Knapsack problems with several knapsacks

Introduction: several knapsacks

With several containers, every item can be assigned to at most one of them. The combinatorial object becomes an assignment.

Definition 5: multiple knapsack problem

Given m knapsacks with capacities c_i, assign each item to at most one knapsack, maximizing profit:

$$\sum_{j \in S_i} w_j \leq c_i \ (i = 1, \ldots, m), \qquad S_i \cap S_h = \emptyset \ (i \neq h).$$

Summary

MKP combines assignment and packing: each selected item has one destination and each knapsack has its own capacity.

Knapsack problems with classes

Introduction: classes and setups

Classes represent mutually exclusive alternatives or families that must be activated before their items can be selected.

Definition 6: multiple-choice knapsack problem

Items are partitioned into classes $\mathscr{J}_h$. Select exactly one item from every class, subject to capacity:

$$|S \cap \mathscr{J}_h| = 1 \quad \forall h, \qquad \sum_{j \in S} w_j \leq c.$$

Definition 7: knapsack problem with setups

Items are partitioned into classes. Selecting an item of class h requires its setup, which consumes setup weight s_h and incurs setup cost f_h.

Summary

- MCKP makes one choice in every class.
- KPS links item choices to setup decisions.
- Both variants anticipate logical constraints in integer programming models.

Typical mistakes

Introduction: mistakes

The variants have similar data and names. Identify what changes in a solution: copies, resources, knapsacks or classes.

Typical mistakes

- Confusing candidate and feasible solutions.
- Treating identical copies as distinguishable in BKP or UKP.
- Assigning an item to two knapsacks in MKP.
- Replacing exactly one per class by at most one in MCKP.
- Forgetting the setup weight or cost in KPS.

Summary

- KP, SSP, BKP, UKP, MDKP, MKP, MCKP and KPS are related but distinct.
- Each variant is defined by its feasible solutions and capacity constraints.
- Their reference instances and terminology return throughout the course.